

PitchSwitch

snig-17

June 27, 2026

CONTENTS

PitchSwitch	
The Problem	
The Solution	
Coach — explainable AI for casual fans	
Technical Approach	
How the Anticipation Model Works	
IBM Granite Integration	
Docling Grounding	
Model Accuracy	
Methodology	
Personalization	
Judging Criteria Mapping	
Quick Start	
Run it locally (30 seconds)	
Rebuild the cache / reproduce the accuracy numbers (optional)	
Future Work	
Built With	
Tests	
Architecture	
Design	
Changelog	

License

PitchSwitch

Like NFL RedZone, but for soccer, and it switches you *before* the goal instead of after.

► [Live demo](#) — pick a favourite team and hit Start. Watch the Director cut to whichever match is heating up and *show you why* (the live danger differential on screen), with IBM Granite calling each switch out loud. Tap “🗣️ commentary” for the spoken call.

The Problem

In the World Cup group stage, the final round of matches in each group kicks off at the exact same time. This isn't an accident. FIFA made it a rule after West Germany and Austria played out a result in 1982 that knocked Algeria out, so now both games in a group run simultaneously and nobody can play to a convenient scoreline.

The catch is you can only watch one screen. Take the 2018 group finale: Germany were playing South Korea while Mexico played Sweden at the same moment, and the two results were tangled together. Pick the Mexico game and you miss South Korea knocking the defending champions out in stoppage time. It happened again in 2022, Japan vs Spain running alongside Germany vs Costa Rica, with the qualifying table flipping three or four times in the last ten minutes across two pitches you couldn't watch at once.

Broadcasters don't really fix this. NFL RedZone works because American football stops between plays and scores constantly, so there are clean moments to cut to. Soccer is the opposite. It flows nonstop and barely scores, so a tool that just switches you to a goal after it goes in is useless, because it shows you the thing you already missed. And smaller nations get almost no airtime at all, even when theirs is the game worth watching.

The Solution

PitchSwitch watches every match at once and tries to get you to the action *before* it happens, not after.

It gives each match a live danger score from the play-by-play data, things like balls carried into the final third, pressure in the box, shots, corners and penalties. The key signal is how fast that danger is rising, which is what catches a game about to boil over instead of one that already has. On the test matches it flagged goals with about 70 seconds of warning on average.

The Director cuts to whichever match's danger is pulling ahead — and shows you *why*: an on-screen danger bar for each match and the differential that triggered the cut. IBM Granite calls each switch in a line a TV presenter would actually say, grounded in real team context parsed with IBM Docling, so the narration knows Son drives Korea's press and Mbappe is France's outlet, not just the score.

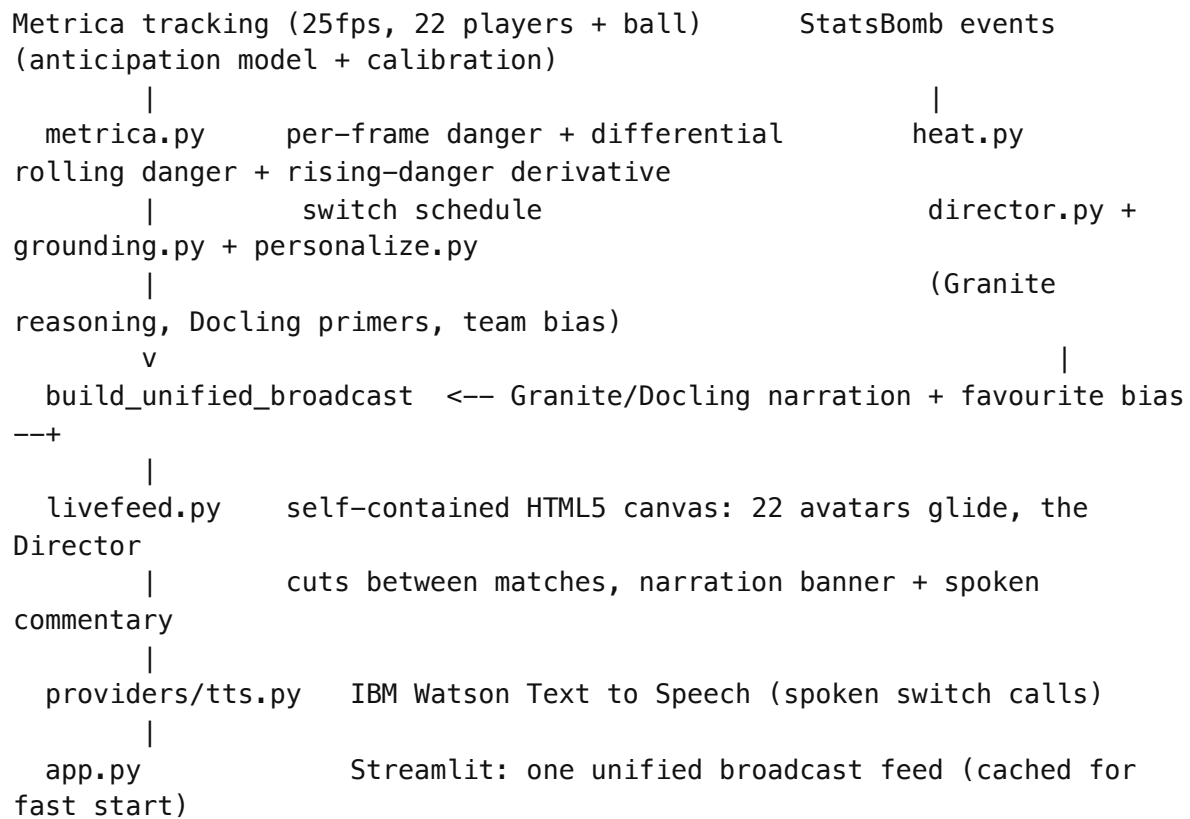
You can name your teams too, including the small nations broadcasters ignore, and it leans the switching toward them.

The main screen is **one live feed over real player tracking** (25fps, all 22 players in national kits with shirt numbers, from Metrica's open data). The Director cuts between matches when one's danger pulls ahead, and Granite calls each switch out loud — spoken in a British commentator voice via **IBM Watson Text to Speech**. It plays like a real broadcast control room, not a dashboard.

COACH — EXPLAINABLE AI FOR CASUAL FANS

World Cup 2026 is in North America, pulling in millions of first-time viewers who don't know offside from a throw-in. So **"Coach"** explains the game as it happens: when a penalty, foul, card, corner, or goal occurs, Coach says the rule in one plain sentence ("Goal: the whole ball crossed the line between the posts and under the bar...") — generated by Granite, **grounded on the actual Laws of the Game parsed by Docling** (`data/rules/`), and spoken in its own voice. It explains only what's deterministically explainable (the rules), never guesses at referee judgment. This makes Docling a backbone, not a decoration, and turns the tool from analyst-facing to audience-facing.

Technical Approach



Three IBM technologies: **Granite** (switch reasoning + narration), **Docling** (team-primer grounding), **Watson Text to Speech** (spoken commentary). The LLM provider is swappable (`providers/llm.py`): Granite runs locally via Ollama, or on IBM watsonx.ai / Replicate in the cloud.

The live feed uses Metrica’s open tracking (real continuous movement, anonymised teams) mapped to World Cup fixtures so Granite/Docling/personalization apply — the team labels are illustrative, a representative stand-in for a licensed broadcast feed (FIFA blocks real match video from embedding). The StatsBomb pipeline (`heat.py` , `scripts/calibrate.py`) backs the anticipation model and the accuracy numbers below.

HOW THE ANTICIPATION MODEL WORKS

Each match maintains a **danger score** (0.0 to 1.0) computed from a rolling 90-second window of events:

- Carries into the final third
- Pressure events in the opponent's half
- Passes into the penalty area
- Shot xG accumulation
- Set-piece indicators (penalty, corner, free kick in range)
- Late-game tight-scoreline multiplier (1.5x in last 15 minutes)

The **forward-looking signal** is the time-based derivative: how fast danger is climbing. A sharp rise triggers the “about to ignite” flag even before the absolute danger peaks. This is what enables switching before the moment, not after.

IBM GRANITE INTEGRATION

In the live broadcast, the switch itself is decided deterministically by the danger differential (the thing you see on screen), so a cut is never blocked on LLM latency. IBM Granite’s job is the *call*: for each switch it generates the natural-language narration a presenter would say — “Switch to Brazil-Croatia, penalty about to be taken after VAR review.” — and it powers Coach’s rule explanations. Both are grounded by Docling.

The repo also ships an event-model Director (`core/director.py`) that uses Granite as a tie-break reasoner when two matches are within 0.15 danger of each other. That path drives the StatsBomb anticipation model used in the offline calibration below; the live tracking feed uses the deterministic differential so the broadcast stays real-time.

DOCLING GROUNDING

Nation-specific primers (team history, key players, tournament context) in `data/primers/*.md` are parsed via IBM Docling into a local knowledge base (`core/grounding.py`). When the Director builds a Granite prompt, it injects the relevant teams’ facts so the narration is grounded in real context: “Switch to South Korea vs Germany - Son Heung-min is leading the Taegeuk Warriors’ famous never-say-die press.”

Docling is optional: if the package isn't installed the app falls back to reading the markdown primers directly, so grounding works either way. The KB loads in a background thread at startup, so the ~6s parse never blocks the replay.

Model Accuracy

Measured across the three demo matches (15 goals) with `scripts/calibrate.py`, using a 120-second lead window:

Metric	Value
Dangerous moments predicted	7/15 (47%)
Average lead time before event	71s
False positive rate	42%

The headline is lead time: ~71 seconds of warning before a goal. That's the whole point — switching you *before* the moment, not after.

METHODOLOGY

`python scripts/calibrate.py [--window SECONDS]` replays each match through the heat model and scores the switch signal (needs `pip install -r requirements-dev.txt`; StatsBomb's open data is fetched via `statsbombpy` on first run and cached locally):

- **Ground truth** = goals; shots (goals + attempts) form the broader “dangerous moment” set used for false-positive scoring, since anticipating a shot or save is also a valid switch.
- **Predicted (recall)**: a goal is predicted if a switch fired for that match in the window *strictly before* the goal — measuring anticipation, not reaction to the goal's own xG spike.
- **False positive**: a switch with no shot/goal within the window after it.

Sensitivity to the window (the model degrades gracefully, not cherry-picked):

Window	Predicted	Avg lead	False positive
90s	4/15 (27%)	44s	49%
120s	7/15 (47%)	71s	42%
180s	9/15 (60%)	98s	27%

Soccer is low-scoring and many goals come from quick counters with little buildup, so perfect recall isn't the goal — surfacing the matches that are *heating up*, early, is. The false-positive rate reflects that most flagged danger (sustained pressure, corners) is genuine even when it doesn't produce a shot within the window.

Personalization

Select your favourite team(s), including small nations that broadcasters ignore. PitchSwitch biases switching toward their matches: “Your team Cape Verde has a corner in a 0-0, get in.”

Judging Criteria Mapping

Criterion	Feature	Evidence
Technical Execution	Anticipation model + 3 IBM techs + real tracking feed	Rolling danger score with forward-looking derivative; Granite reasons over structured state, Docling grounds the narration, Watson TTS speaks it; live 25fps tracking broadcast
Innovation	Multi-match anticipation + small-nation personalization	Uncontested: no tool predicts and switches pre-event; small-nation bias is unique
Challenge Fit	"AI Inside the Match" + World Cup specific	Dead-on theme; structurally WC-specific (simultaneous group-stage games)
Implementation & Feasibility	Free data, swappable LLM, clear product path	StatsBomb free data; provider interface for watsonx/Replicate/Ollama; 10x = broadcast control room API

Quick Start

No install needed to try it — open the [live demo](#).

RUN IT LOCALLY (30 SECONDS)

The broadcast cache is committed to the repo, so the demo runs with nothing but the runtime deps — **no data download, no Ollama, no API keys.**

```
git clone https://github.com/snig-17/pitchswitch.git
cd pitchswitch
pip install -r requirements.txt
streamlit run app.py          # pick a team, hit Start – it serves the pre-
                               built cache
```

REBUILD THE CACHE / REPRODUCE THE ACCURACY NUMBERS (OPTIONAL)

Only needed to regenerate broadcasts with fresh Granite narration, or to re-run the accuracy calibration. This is the heavier path (ML deps, tracking data, local LLM):

```

pip install -r requirements-dev.txt          # statsbompy, docling, numpy,
mplsoccer
bash scripts/get_metrica.sh                 # Metrica tracking data
(~120MB, gitignored)
cp .env.example .env                       # configure providers
ollama pull granite3.3:2b                  # local IBM Granite for
narration
# optional: add IBM Watson Text to Speech creds in .env for spoken
commentary

python scripts/prebuild_cache.py            # rebuild the broadcast cache
python scripts/calibrate.py                # reproduce the Model Accuracy
table above

```

A rebuild runs Granite (~30–40s per favourite team) and caches the result, so the app then loads each broadcast in ~1s.

Future Work

- Connect to real-time match feeds (Opta, Stats Perform) for live World Cup 2026
- Broadcast control room API: feed anticipation signals to human directors
- Social watch parties with combined team preferences
- Mobile companion app

Built With

- **IBM Granite** (switch reasoning + narration)
- **IBM Docling** (grounds Granite: team primers + the Laws of the Game for Coach)
- **IBM Watson Text to Speech** (spoken commentary)
- **Metrica Sports** open data (25fps player tracking)
- **StatsBomb** open data (event data: anticipation model + accuracy calibration)

- **mplsoccer / Streamlit / HTML5 canvas** (the live feed + UI)
- **Claude Code** (development tool)

Tests

`pytest` runs the unit suite over the live path (danger model, switching bias, Coach, and a heat-model regression) — green in CI on every push. See [TESTING.md](#).

Architecture

The two-engine design (calibrated StatsBomb model vs the live tracking feed), data flow, and where the IBM tech sits is explained in [ARCHITECTURE.md](#).

Design

The visual language (broadcast control-room theme, tokens, type, component vocabulary) is documented in [DESIGN.md](#).

Changelog

See [CHANGELOG.md](#) for the build history.

License

MIT — see [LICENSE](#). Third-party open data (StatsBomb, Metrica Sports) is under its own respective terms.